

Exercise Set 6

1. Readers/Writers with priorities (given Python code)

1(a) Starvation in the given solution

Can a writer starve? Yes. Writers wait while `waitingreaders > 0` (and also while any readers are active). New readers are allowed to enter whenever there is no active writer (`len(writers)==0`). If readers keep arriving, `waitingreaders` may rarely drop to 0, so writers can be postponed indefinitely.

Can a reader starve? No. A reader only waits while there is an active writer (`len(writers) > 0`). When a writer finishes, if there are waiting readers, it wakes them (`oktoread.notify_all()`), and readers do not wait behind waiting writers.

1(b) Who has higher priority and why?

Readers have higher priority. Reason: writers are blocked not only by active readers/writers, but also by waiting readers:

```
while len(writers) + len(readers) + waitingreaders > 0:
```

So even if no one is currently reading, queued readers prevent writers from starting.

1(c) Modify the code so that writers have priority

Writer-priority policy: - A reader waits if a writer is active OR if any writers are waiting. - When finishing, wake writers first.

```

import threading

readers = set()
writers = set()
waitingreaders = 0
waitingwriters = 0

lock = threading.Lock()
oktoread = threading.Condition(lock)
oktowrite = threading.Condition(lock)

def reader():
    global waitingreaders, waitingwriters
    name = threading.current_thread().name
    for _ in range(3):
        with lock:
            while len(writers) > 0 or waitingwriters > 0:
                waitingreaders += 1
                oktoread.wait()
                waitingreaders -= 1
            readers.add(name)
            print(name + ": Starts reading")
            print("Current readers: " + str(readers) + " Current writers: " + str(writers))
        print(name + ": Reading...")
        with lock:
            print(name + ": Finishes reading")
            readers.remove(name)
            if len(readers) == 0 and waitingwriters > 0:
                oktowrite.notify()

def writer():
    global waitingreaders, waitingwriters
    name = threading.current_thread().name
    for _ in range(3):
        with lock:
            while len(readers) > 0 or len(writers) > 0:
                waitingwriters += 1
                oktowrite.wait()
                waitingwriters -= 1
            writers.add(name)
            print(name + ": Starts writing")
            print("Current readers: " + str(readers) + " Current writers: " + str(writers))

```

```

print(name + ": Writing...")
with lock:
    print(name + ": Finishes writing")
    writers.remove(name)
    if waitingwriters > 0:
        oktowrite.notify()
    elif waitingreaders > 0:
        oktoread.notify_all()

```

1(d) Starvation in the writer-priority solution

Can a reader starve? Yes. If writers keep arriving, `waitingwriters > 0` remains true and readers keep waiting.

Can a writer starve? No. Once any writer is waiting, new readers stop entering, and writers will eventually proceed.

2. Deadlock

Threads (from the sheet): - T1: get(A) get(B) get(C) - T2: get(D) get(E) get(B) - T3: get(C) get(F) get(D)

2(a) Show possibility of deadlock (cycle)

One deadlocking interleaving: 1. T1 acquires A, then B 2. T3 acquires C, then F 3. T2 acquires D, then E 4. Next requests:

- T1 requests C (held by T3)
- T3 requests D (held by T2)
- T2 requests B (held by T1)

Cycle: T1 holds B and waits for C T3 holds C and waits for D T2 holds D and waits for B So deadlock is possible.

2(b) Reorder gets inside threads to prevent deadlock

Impose a global resource order, e.g. $A < B < C < D < E < F$, and request resources only in increasing order: - T1: A, B, C - T2: B, D, E - T3: C, D, F

This prevents cycles in the wait-for graph.

3. SSDs (OSTEP Chapter 44, questions 1-5)

The questions and simulator usage are in the OSTEP SSD chapter homework.

3.1 (Q1) -T log -s 1 -n 10 -q

This produces 10 random operations against a log-structured SSD. With -q, commands are hidden, so infer them from mapping/validity changes. Use -c or -C to reveal the exact operations and verify.

What to submit: - The 10 commands produced by seed 1 (from -c or -C) - Short reasoning showing how each affects mapping and page validity

3.2 (Q2) -T log -s 2 -n 10 -C

Commands are shown (-C). Track after each command: - logical-to-physical mapping updates - valid/invalid physical pages - log write pointer movement Verify with -F which shows flash state between commands.

3.3 (Q3) Add -r 20

Higher read fraction means more reads and fewer writes/trims compared to Q2, so less invalidation and less pressure on free space. Verify the command mix with -c.

3.4 (Q4) Estimate completion time for -T log -s 1 -n 10

Total time is approximately: $T = 1000(\#erases) + 40(\#programs) + 10*(\#reads)$ microseconds
Then verify with -S statistics output.

3.5 (Q5) Compare -T log vs -T direct

Direct updates are expensive on flash because overwrites require erases (and often extra copying), typically increasing erase count and time. Log-structured appends and invalidates old pages, often reducing erases. Confirm by comparing -S totals for both runs.

4. UNIX file management (inode addressing)

Given (from the sheet): - Block size = 8 KB = 8192 bytes - 12 direct pointers - single, double, triple indirect - pointer size = 4 bytes - pointer has 8-bit disk id + 24-bit block id

Pointers per indirect block: $8192 / 4 = 2048$ pointers

4(a) Maximum file size

Data blocks addressable: - direct: 12 - single: 2048 - double: $2048^2 = 4,194,304$ - triple: $2048^3 = 8,589,934,592$ Total blocks = $12 + 2048 + 4,194,304 + 8,589,934,592 = 8,594,130,956$

Max file size: $8,594,130,956 * 8192 = 70,403,101,663,232$ bytes 64.03 TiB

4(b) Maximum system partition size

24-bit block id per disk: $2^{24} = 16,777,216$ blocks Disk size: $16,777,216 * 8192 = 137,438,953,472$ bytes = 128 GiB So max physical disk (partition) size is 128 GiB.

4(c) Disk accesses to byte position 13,423,956

Block index: $13,423,956 // 8192 = 1,638$ Offset: $13,423,956 \bmod 8192 = 5,460$

Direct blocks cover indices 0-11. Single indirect covers indices 12-2059. 1638 is in single-indirect range.

If only the inode is in memory: 1. read the single-indirect pointer block 2. read the data block
Total disk accesses: 2

5. Storing one file per network packet

5(a) Disadvantages - Massive metadata overhead (inode + dir entry per packet) - High syscall and fsync/open/close overhead per packet - Directories become huge and slow to list/search/backup - Fragmentation and poor locality from many tiny writes - Analytics is slower than streaming from a single structured log

5(b) Why the system could crash - Running out of inodes even if disk space remains - Disk fills due to metadata + journaling overhead - File system/kernel stress from huge directory trees and metadata churn - Hitting system limits (e.g., too many open files)

5(c) Better organization - Append packets into large rotating log files (time-based or size-based) - Use standard capture formats like pcap/pcapng or a simple binary log - Store logs in time-hierarchy folders (YYYY/MM/DD/HH) to keep dirs small - Compress older chunks and maintain an index for lookup